IIT MADRAS · SOFTWARE ENGINEERING · SEPT 2026

Milestone 2 Report

Scheduling, Design and Frontend

Student Mentoring and Career Support Platform

Problem Statement 2

Team 032 – Himalayan Coders

Himanshu Rathor · Ankur Maurya · Hemendra Kumar Sharma
Adnan Mustafa · Bojannagari Pavan Kumar Reddy

1. Project Schedule and Sprint Plan

1.1 Scheduling tool

The team plans and tracks its work in **Jira** (project key KAN). Epics group the work of each milestone or sprint. Tasks carry one owner, a start date and a due date, and move across the board columns To Do, In Progress, In Review and Done. The Jira **Timeline** view is our Gantt chart and the **Board** view is our Kanban board. Each task records its owner, so both views show every member's contribution.

1.2 Team roles

Member	Roles
Himanshu Rathor	Scrum Master, Product Manager
Ankur Maurya	Backend Developer, Frontend Developer
Hemendra Kumar Sharma	Frontend Developer, Backend Developer
Adnan Mustafa	Tester, Code Reviewer
Bojannagari Pavan Kumar Reddy	Code Reviewer, Tester

1.3 Overall schedule

Phase	Dates	Focus
Milestone 1	until 25 Oct	User requirements
Milestone 2	26 Oct – 1 Nov	Scheduling, design, frontend pages
Sprint 1 (Milestone 3)	2 Nov – 22 Nov	APIs, test cases and user testing for the core flow
Sprint 2 (Milestone 4)	23 Nov – 6 Dec	APIs, test cases and user testing for the rest of the flow
Milestone 5	7 Dec – 20 Dec	Final report, demo video, working prototype

1.4 Sprint 1 (2 Nov – 22 Nov): core client-mediated flow

Jira task	User stories	Dates
Registration and login APIs	US-S01, US-M01	2 – 8 Nov
Student and mentor profile APIs	US-S02, US-M02, US-C05	6 – 12 Nov
Admin dashboard and role-based access APIs	US-C01, US-C06	9 – 15 Nov
Match creation APIs	US-C02	12 – 18 Nov
Test cases, pytest and Swagger YAML	all of the above	9 – 20 Nov
User feedback session and Sprint 2 plan	–	20 – 22 Nov

1.5 Sprint 2 (23 Nov – 6 Dec): confirmation, tracking and student views

Jira task	User stories	Dates
Match confirmation and notification APIs	US-C03	23 – 29 Nov
Student goals, preferences and availability APIs	US-S03 to US-S06	23 – 29 Nov
Match status management and overview APIs	US-C04, US-C07	26 Nov – 2 Dec
Student connection view, readiness, concern, feedback and status APIs	US-S07 to US-S11	28 Nov – 3 Dec
Test cases, pytest and Swagger YAML	all of the above	30 Nov – 5 Dec
User feedback session and final-phase plan	–	4 – 6 Dec

All 21 user stories from Milestone 1 are covered across the two sprints.

1.6 Iterations

Each sprint is split into weekly iterations. Every sprint ends with a user feedback session whose findings shape the next sprint plan. Tasks are reviewed by the Code Reviewers and tested by the Testers before they move to Done.

1.7 Gantt chart

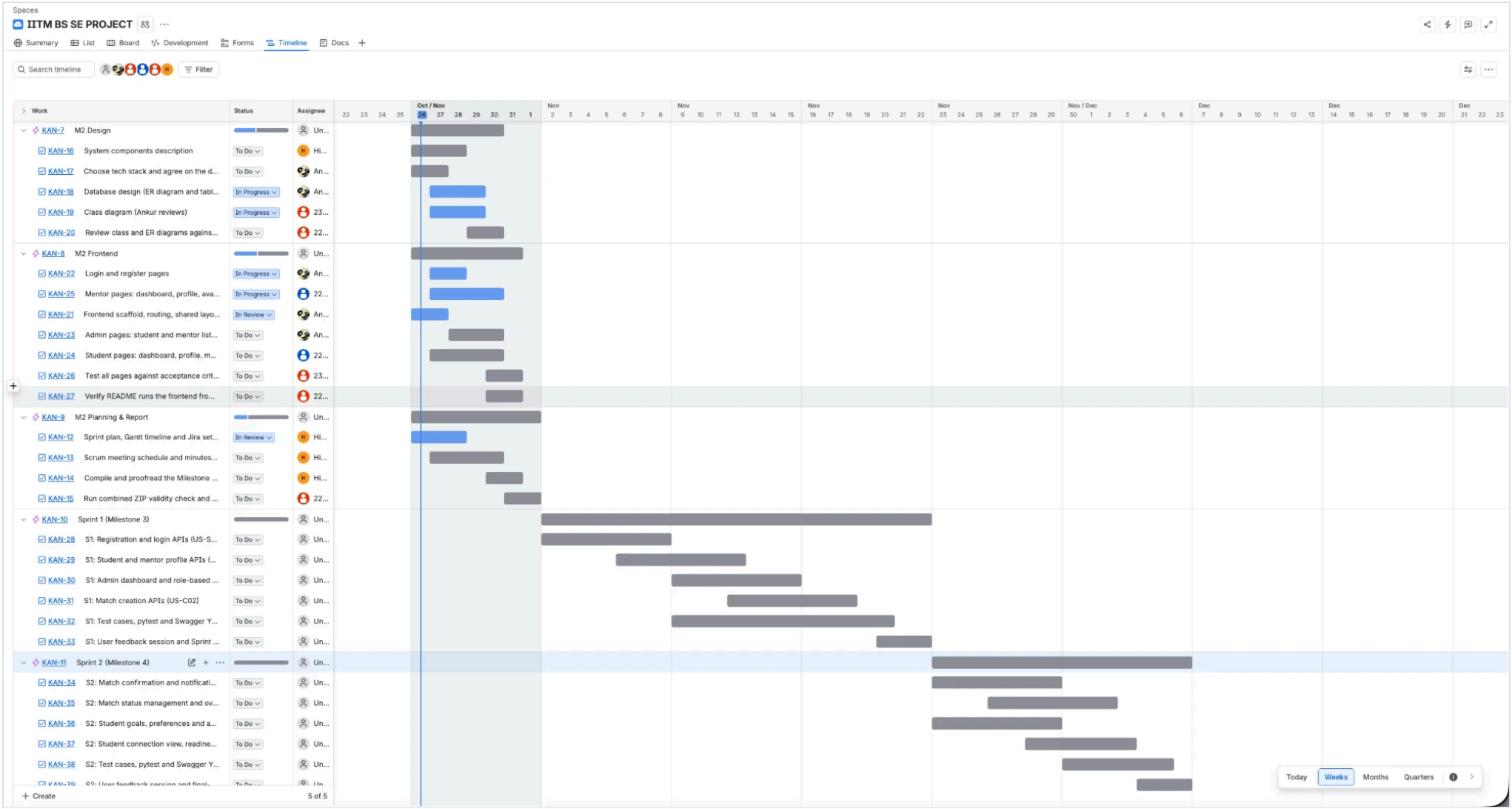


Figure A. Jira Timeline (Gantt) with the epics expanded, from 26 Oct to December.

1.8 Kanban board

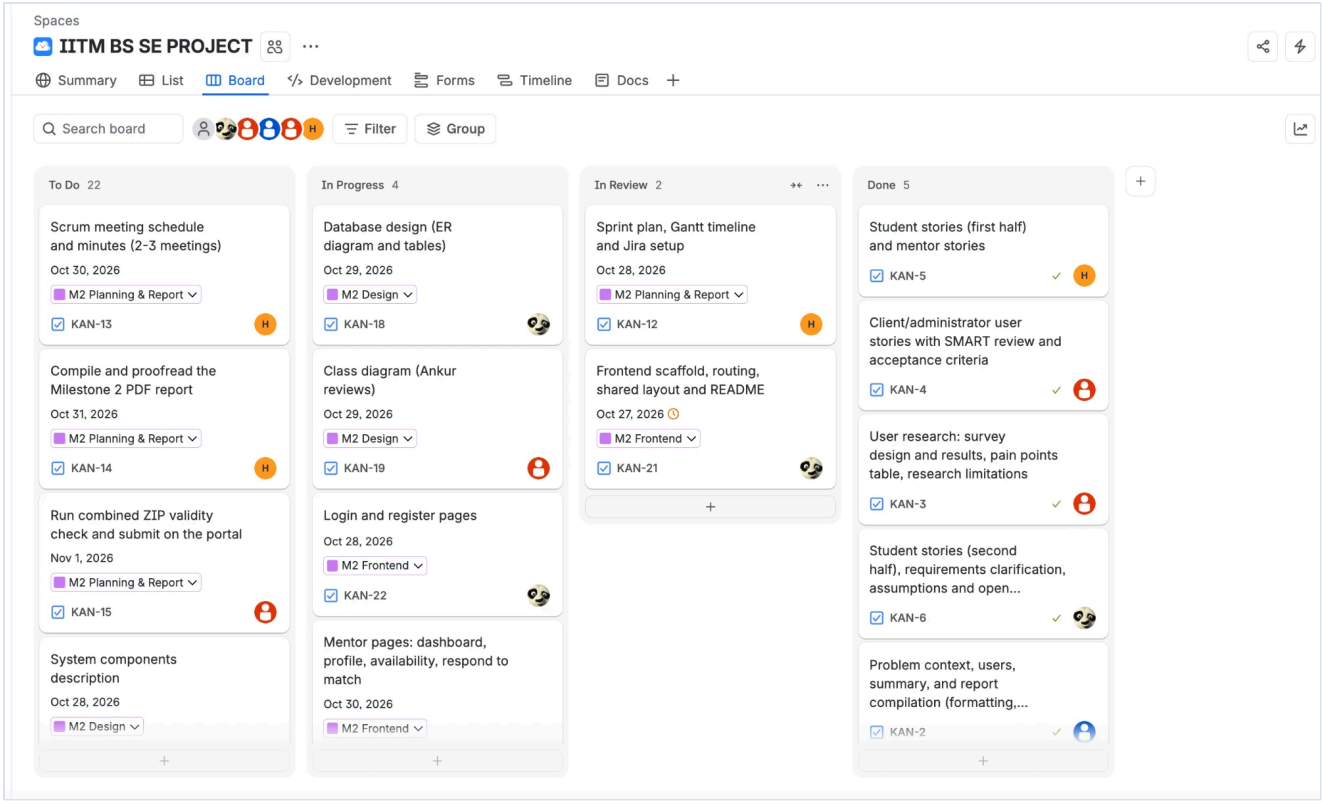


Figure B. Jira Kanban board with the Milestone 2 tasks across To Do, In Progress, In Review and Done.

2. Scrum Meetings Schedule and Minutes

Scrum meetings: held about twice a week on Google Meet and led by the Scrum Master. Each meeting covers what was done, what is planned next and any blockers. Two meetings were held during Milestone 2.

Minutes from Scrum Meeting 1

Date and time	27 October 2026, 6:00 – 6:57
Mode	Google Meet
Attendees	Ankur, Adnan, Hemendra, Himanshu, Pavan
Agenda	Frontend design; database schema; workflow for matching students to mentors.

Discussion. The team discussed the overall design of the frontend and the database schema of the platform. The team also discussed how a student is matched to a mentor. Each match will show a status of Proposed, Active, Completed or Cancelled.

Decision. The match status values are fixed as Proposed, Active, Completed and Cancelled.

Next meeting agenda. Continue discussing the frontend design in more detail, and decide how the dashboards will look for the Admin, the Student and the Mentor.

Minutes from Scrum Meeting 2

Date and time	31 October 2026, 7:00 – 7:45 PM
Mode	Google Meet
Attendees	Ankur, Adnan, Hemendra
Agenda	Scrum call 2 minutes of meeting.

#	Discussed	Action (owner)
1	Jira tasks and dashboard.	The team reviews their tasks and makes changes.
2	Changes in the ER diagram.	Ankur (completed)
3	Milestone 2 report review.	Adnan
4	User story US-C03 clarification: the design was modified as per this story.	Ankur (completed)
5	GitHub Milestone 2 review.	The team, as per the tasks assigned in Jira

3. Design of Components

The platform follows a client-mediated model. Students and mentors each maintain their own profile, but they cannot browse each other. The client (coordinator) reviews both sides, proposes a match, and tracks it until it ends.

Component	Responsibilities
Authentication and roles	Registration, login, and three roles: student, mentor and administrator. Every action is checked against the role on the server, because hiding a link in the interface is not security.
Student module	Maintain goals, skills, wanted topics (chosen from a shared list), preferred format and duration, and free times (day and time band). See the status and next step of the match. Accept or decline the mentor proposed by the coordinator (the answer is stored with its date), view the mentor once the match is active, give feedback, and raise concerns.
Mentor module	Maintain qualifications, topics (from the shared list), bio, availability slots and the maximum number of students they can mentor at the same time. Accept or decline match requests (the answer is stored with its date) and see active mentees.
Administrator module	View and search student profiles, review, edit and verify mentor profiles, see scored match recommendations, propose matches with the student's needs, the mentor's offer and the score side by side, confirm a match once both sides have accepted or cancel it, change match status, review concerns and student feedback, and see delayed proposals and an overview with profile counts and matches by status.
Matching and recommendation	Scores every student and mentor pair out of 100, hides pairs that are not allowed (mentor not verified, mentor full, student already matched), and recommends the best mentors to the coordinator with the reasons. It only suggests; the coordinator decides. See section 3.1.
Match workflow	Controls the status of a match: proposed, active, completed or cancelled. A match can be confirmed (made active) only after both the mentor and the student have accepted. Only allowed transitions are accepted and each change is logged.
Notifications	Tells both the mentor and the student when a match is proposed, tells the coordinator when each side accepts or declines and when both have accepted, and tells both people when the coordinator confirms or cancels, with the agreed next step.
Data layer	A relational database holding users, profiles, the shared topic list, availability of students and mentors, matches with their scores, status log, feedback, concerns and notifications.

Match status flow: the coordinator proposes a match and both the mentor and the student are notified. Each accepts or declines. Only when **both have accepted** can the coordinator confirm the match, which makes it active; the coordinator can instead cancel it at any time. An active match is later completed or cancelled. If a side declines, the coordinator cancels the match and can propose a new one.

3.1 How matches are recommended

The coordinator should not have to compare every student with every mentor by eye. The platform therefore prepares a short list. The method is simple and explainable, with no machine learning: it only adds points for facts the coordinator can see.

Part of the score	Max points	How it is worked out
Topics	40	The share of the topics the student wants that the mentor also covers. Topics are chosen from one shared list, so both sides use the same words.
Time overlap	30	The share of the student's free day-and-time-band slots (morning, afternoon, evening) that fall inside one of the mentor's availability slots.
Free places	15	Free places divided by the mentor's maximum. Mentors with more room score higher, which spreads the load.
Experience	15	Years of experience, counted up to 10 years.

Pairs that are never recommended: the mentor is not verified; the mentor already has as many proposed or active matches as the maximum they set in their profile; the student already has an open (proposed or active) match. Cancelled and completed matches free the place. A score below 40 is shown as a weak fit, and the coordinator may still propose it.

Example from the sample data: Neha Joshi (wants Product Management and Career Switching, free Sunday and Wednesday evening) and Priya Nair (covers both topics, available Sunday 17:00–19:00 and Wednesday 18:00–19:30, 2 free places, 8 years of experience) score 40 + 30 + 15 + 12 = **97**. For Rohan Das, Priya scores 62 and Dr. Meera Iyer 25 (a weak fit), so Priya is recommended first. Each row also shows the reasons, for example "Shared topics: Interview Preparation; Overlapping time: Sunday evening".

Mentor capacity: each mentor sets "maximum students at the same time" (1 to 10, default 3) in their profile. The dashboard shows how many places are used. A proposal that would go over the maximum is refused.

Coordinator dashboard: the overview page is arranged by urgency. At the top are five cards with counts (matches ready to confirm, proposals declined by a side, mentors to verify, students waiting for a match, open concerns), each linking to the page where the work is done. Below are the recommended matches with a Propose button, the capacity of every mentor, the pipeline by status, concerns and feedback. The Matches page shows the top three mentors for every waiting student. When a pair is proposed, the score and its reasons are saved with the match. The scoring logic is built in Sprint 1 (match creation APIs, KAN-31).

4. Class Diagram

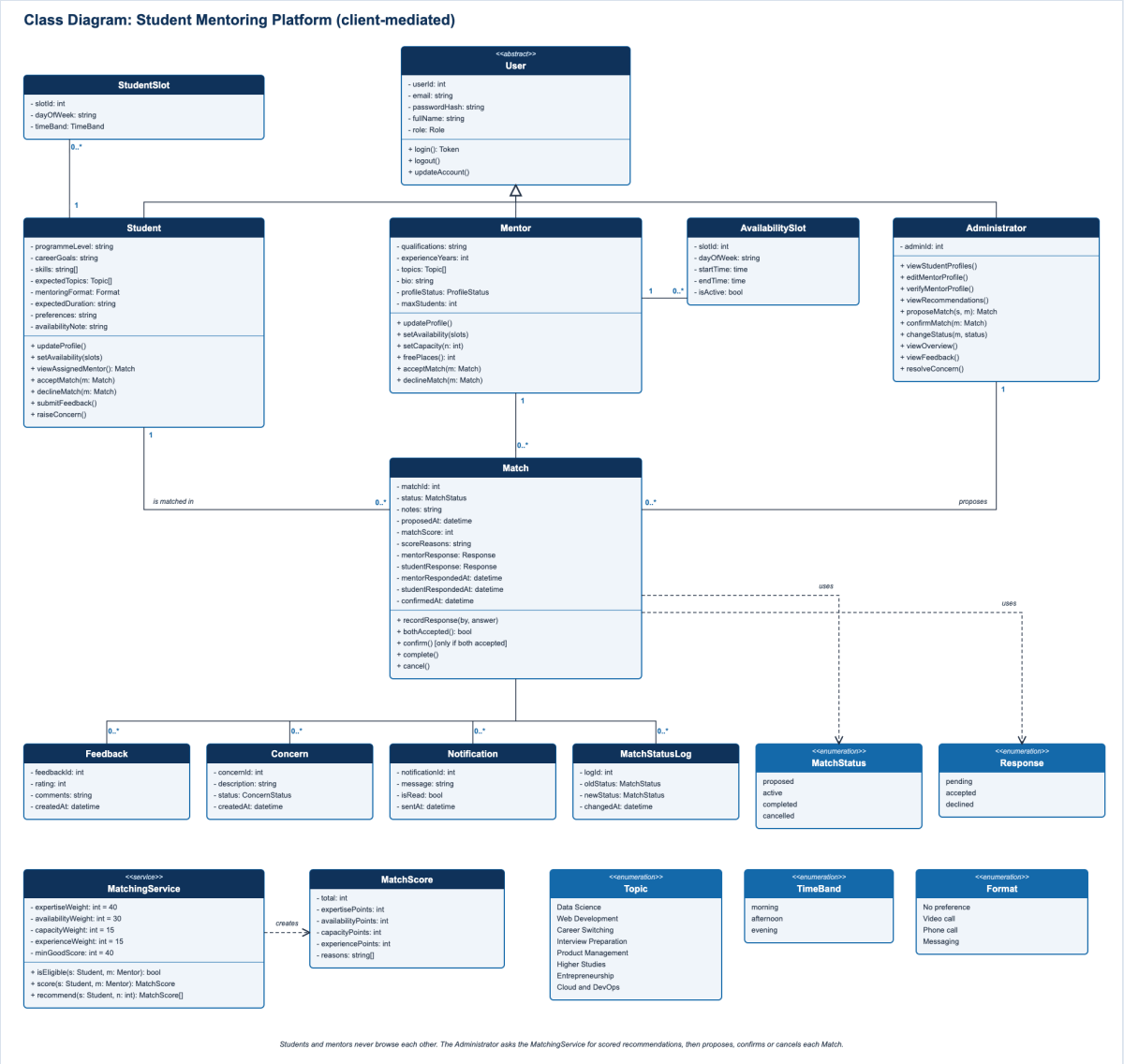


Figure 1. Class diagram. Student, Mentor and Administrator extend User. Every Match is proposed by an Administrator and links one Student and one Mentor. The MatchingService scores pairs and creates MatchScore objects for the coordinator.

5. Database Design

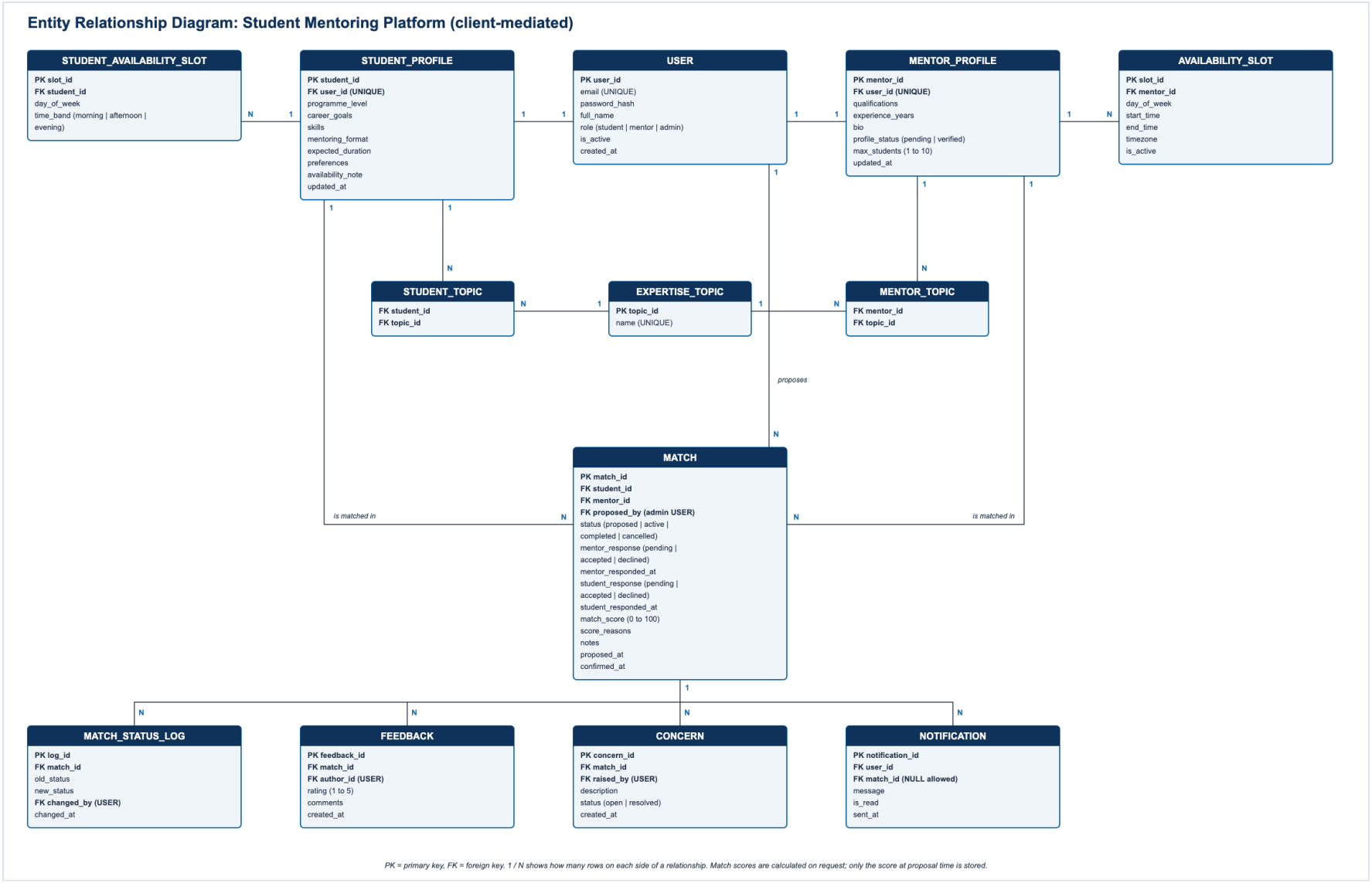


Figure 2. Entity relationship diagram.

Table	Purpose and key constraints
USER	One row per account. Email is unique. Role is student, mentor or admin.
STUDENT_PROFILE	Goals, skills, mentoring format and duration, preferences and an availability note. One per student user (user_id unique).
STUDENT_AVAILABILITY_SLOT	Free times of a student: a day and a time band (morning, afternoon, evening). Many per student. Used for the time-overlap score.
EXPERTISE_TOPIC	The shared list of topics (for example Data Science, Web Development). The topic name is unique.
STUDENT_TOPIC, MENTOR_TOPIC	Link tables: the topics a student wants and the topics a mentor covers (many to many). Used for the topic score.
MENTOR_PROFILE	Qualifications, experience, bio, verification status and `max_students`, the number of students the mentor can handle at once. One per mentor user.
AVAILABILITY_SLOT	Weekly time slots of a mentor. Many slots per mentor.
MATCH	Links a student and a mentor. Records the administrator who proposed it, its status, the mentor's and the student's response with dates, the match score and its reasons at the time of proposal, and when it was confirmed.
MATCH_STATUS_LOG	History of every status change, with who changed it and when.
FEEDBACK	Rating and comments about a match.
CONCERN	A concern raised about a match, open until the coordinator resolves it.
NOTIFICATION	Messages sent to users, optionally linked to a match.

6. Frontend Pages and Screenshots

The frontend is a set of 11 linked pages in plain HTML, CSS and JavaScript. It runs from the file `index.html` without installing anything. Sample data is kept in the browser so the whole flow can be tried: the coordinator proposes a match, the mentor accepts, and only then does the student see the mentor. There is no backend yet; the API is built in Milestones 3 and 4.

Page	Role	Purpose
index, register	all	Log in and create an account

student	student	Dashboard, match status, notifications
student-profile	student	Goals, skills, wanted topics, format, duration, free times
student-mentor	student	Status and next step, proposed mentor to accept or decline, feedback, concerns
mentor	mentor	Accept or decline requests, active mentees
mentor-profile	mentor	Background, topics, capacity and availability
admin	coordinator	Action cards, recommended matches, mentor capacity, pipeline, concerns, feedback
admin-students	coordinator	View, search and filter student profiles and responses
admin-mentors	coordinator	Review, edit and verify mentor profiles, availability and capacity
admin-matches	coordinator	Recommended matches with scores, compare student and mentor, propose, confirm or cancel, change status

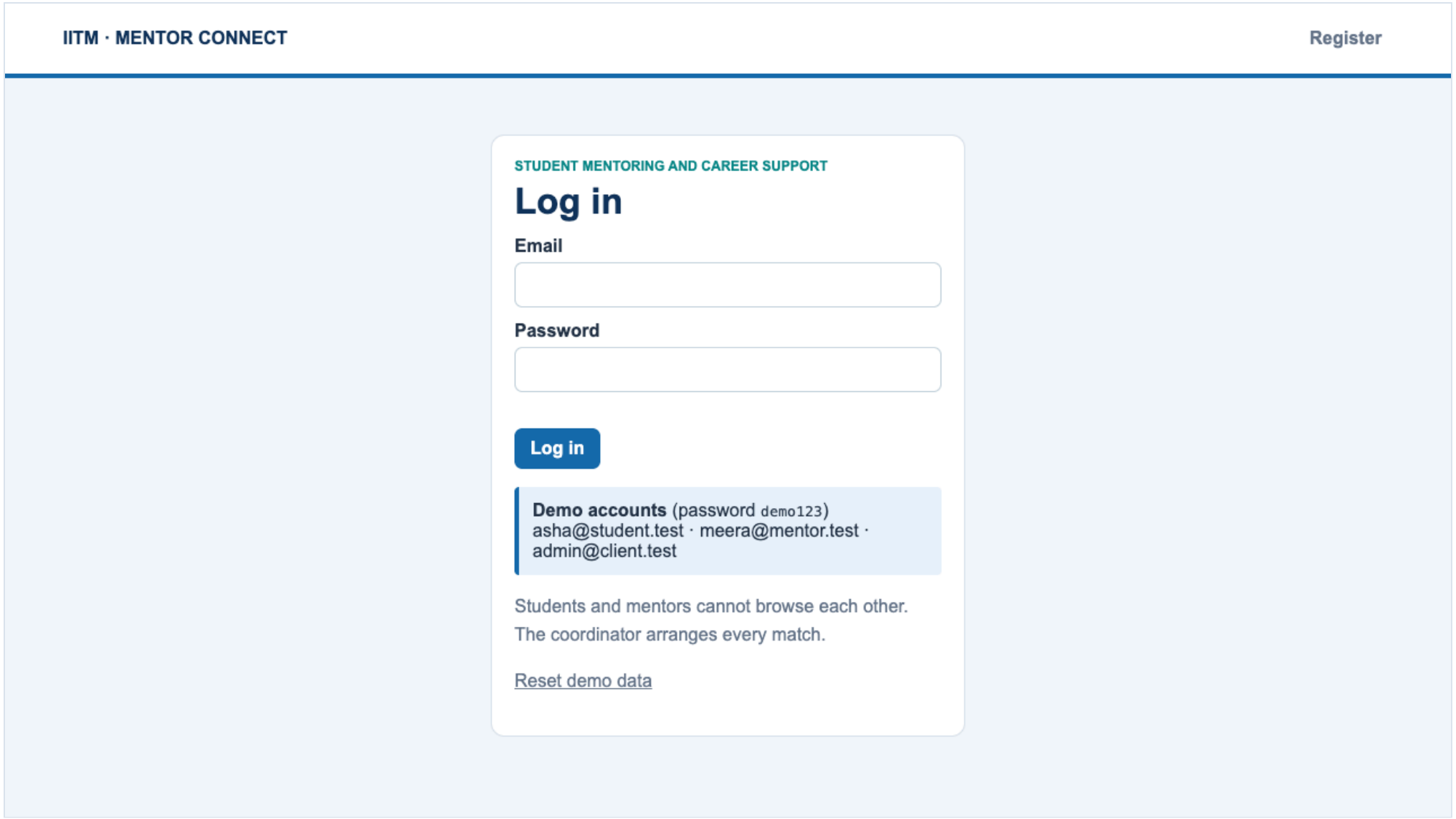


Figure 3. Login page with demo accounts.

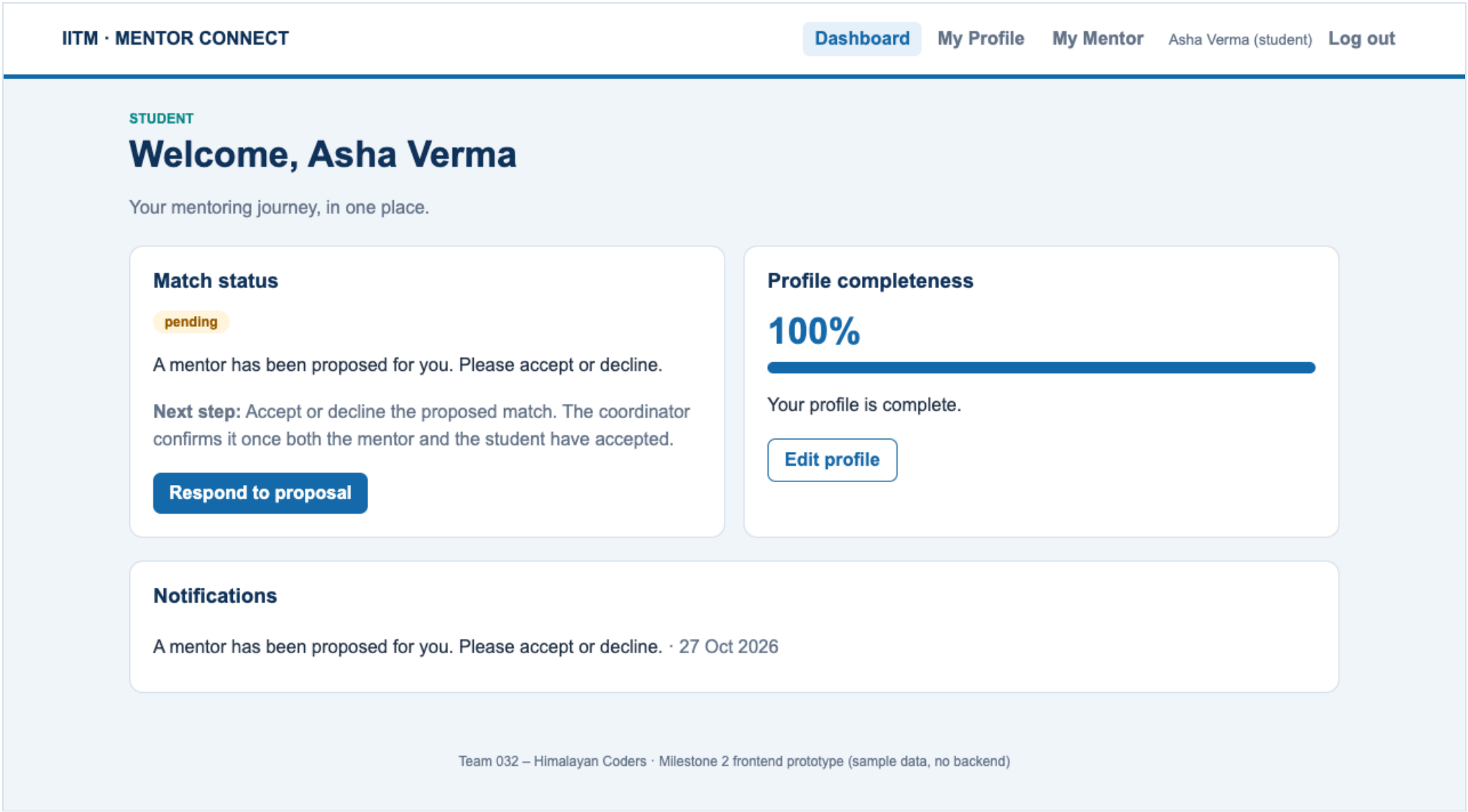


Figure 4. Student dashboard.

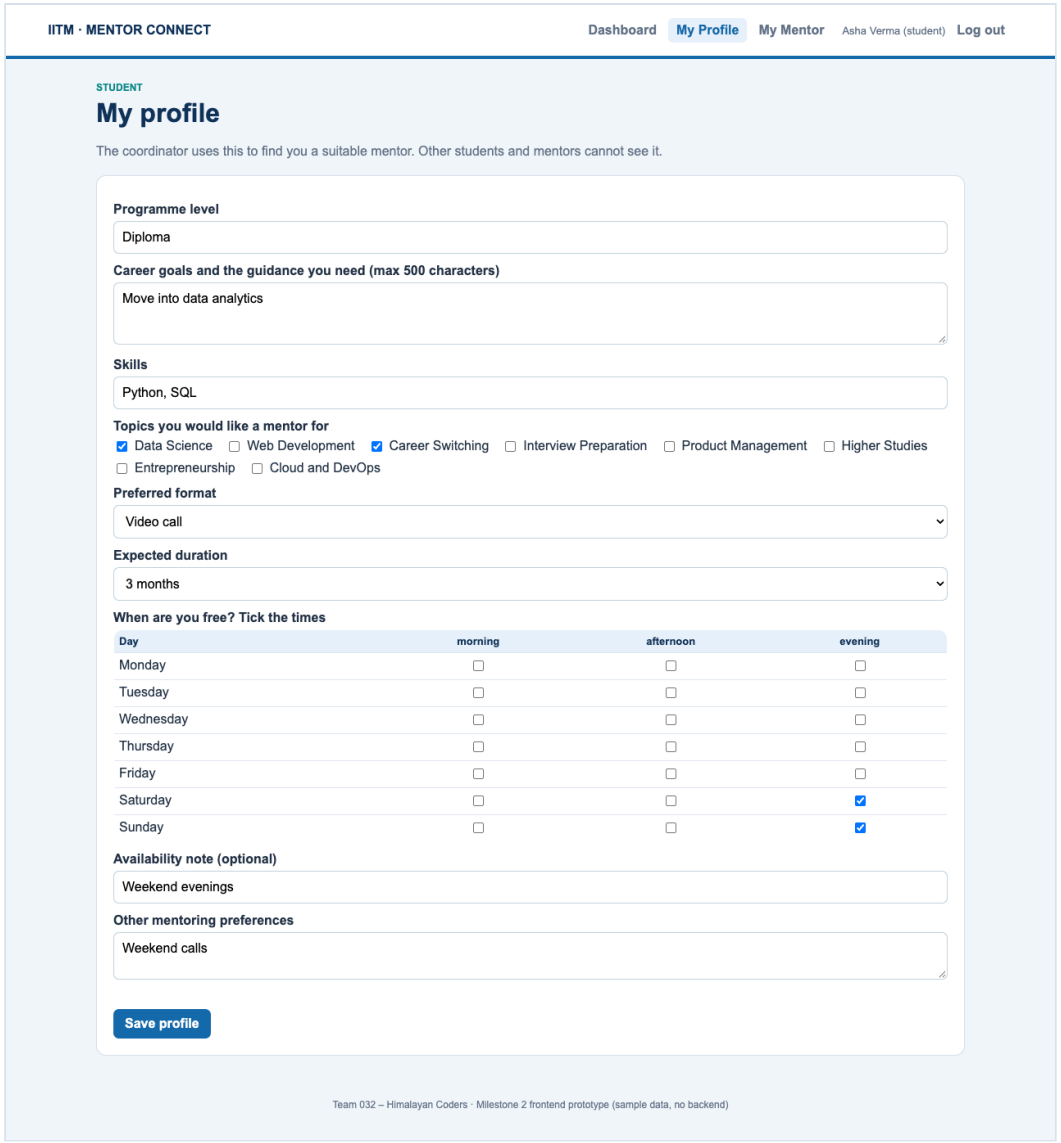


Figure 5. Student profile: wanted topics from the shared list, format, duration and a grid of free times.

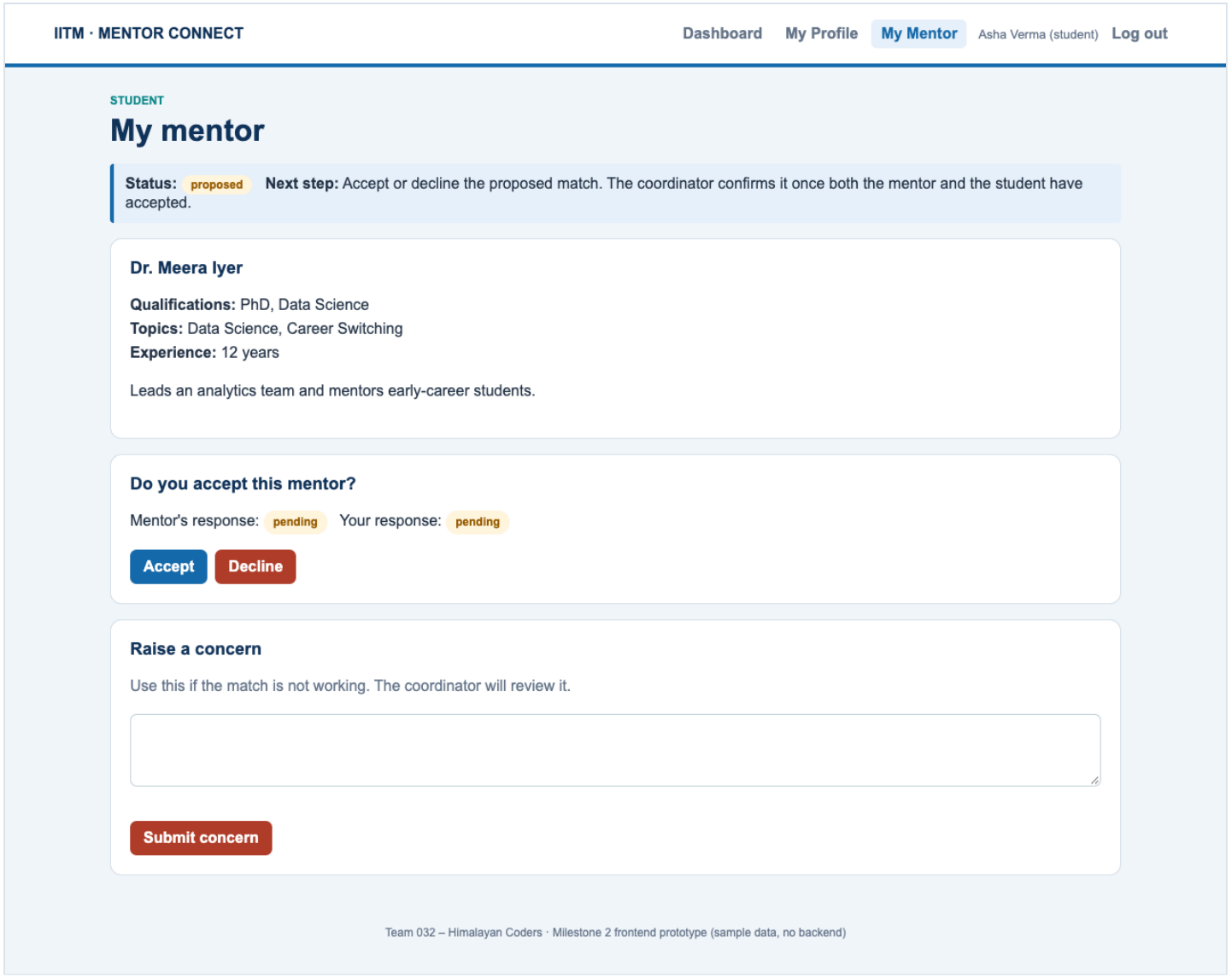


Figure 6. Student's mentor page while a match is proposed: the proposed mentor and the Accept and Decline buttons.

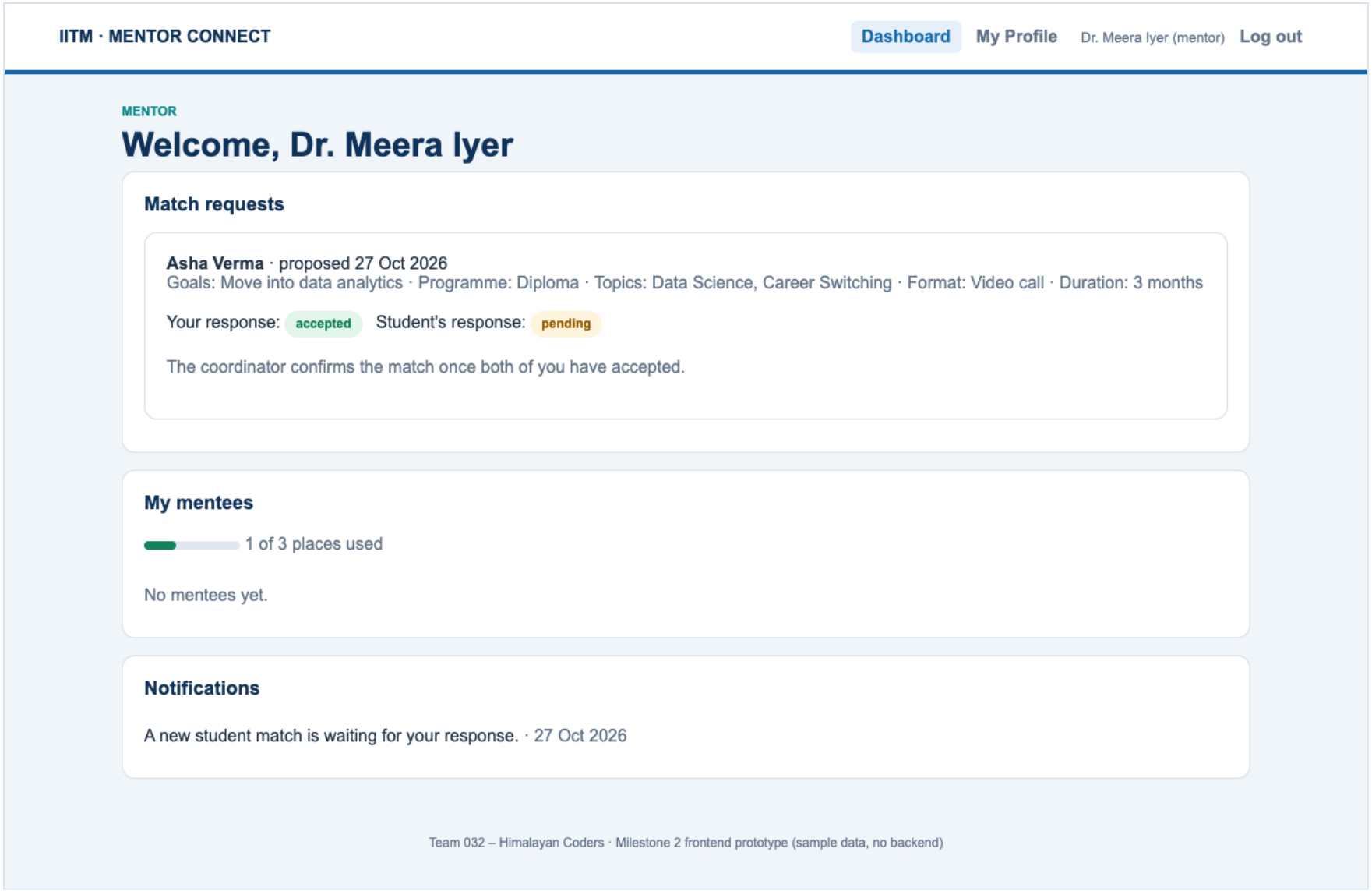


Figure 7. Mentor dashboard with a match request to accept or decline, showing both responses.

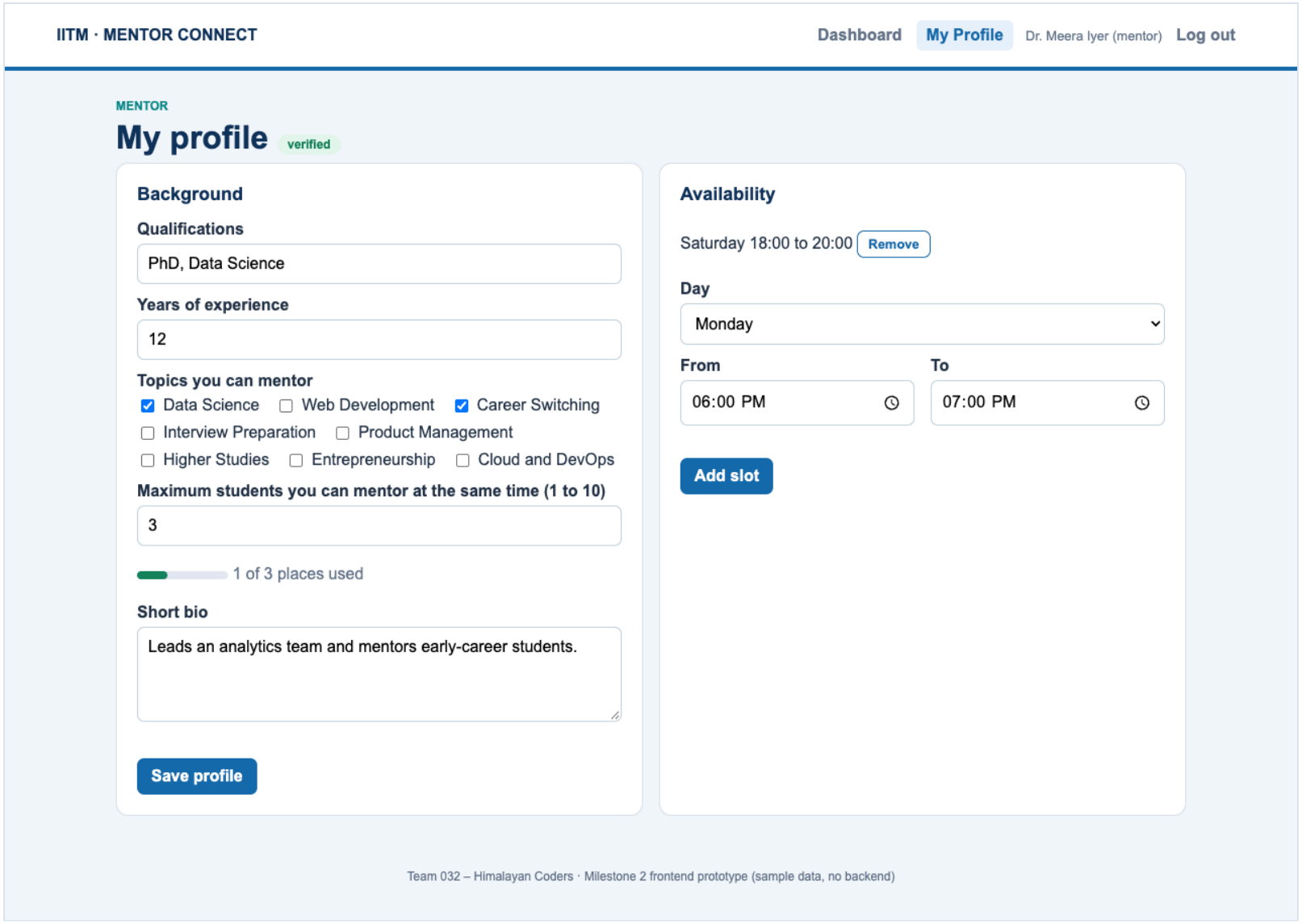


Figure 8. Mentor profile: topics, the maximum number of students, and availability slots.

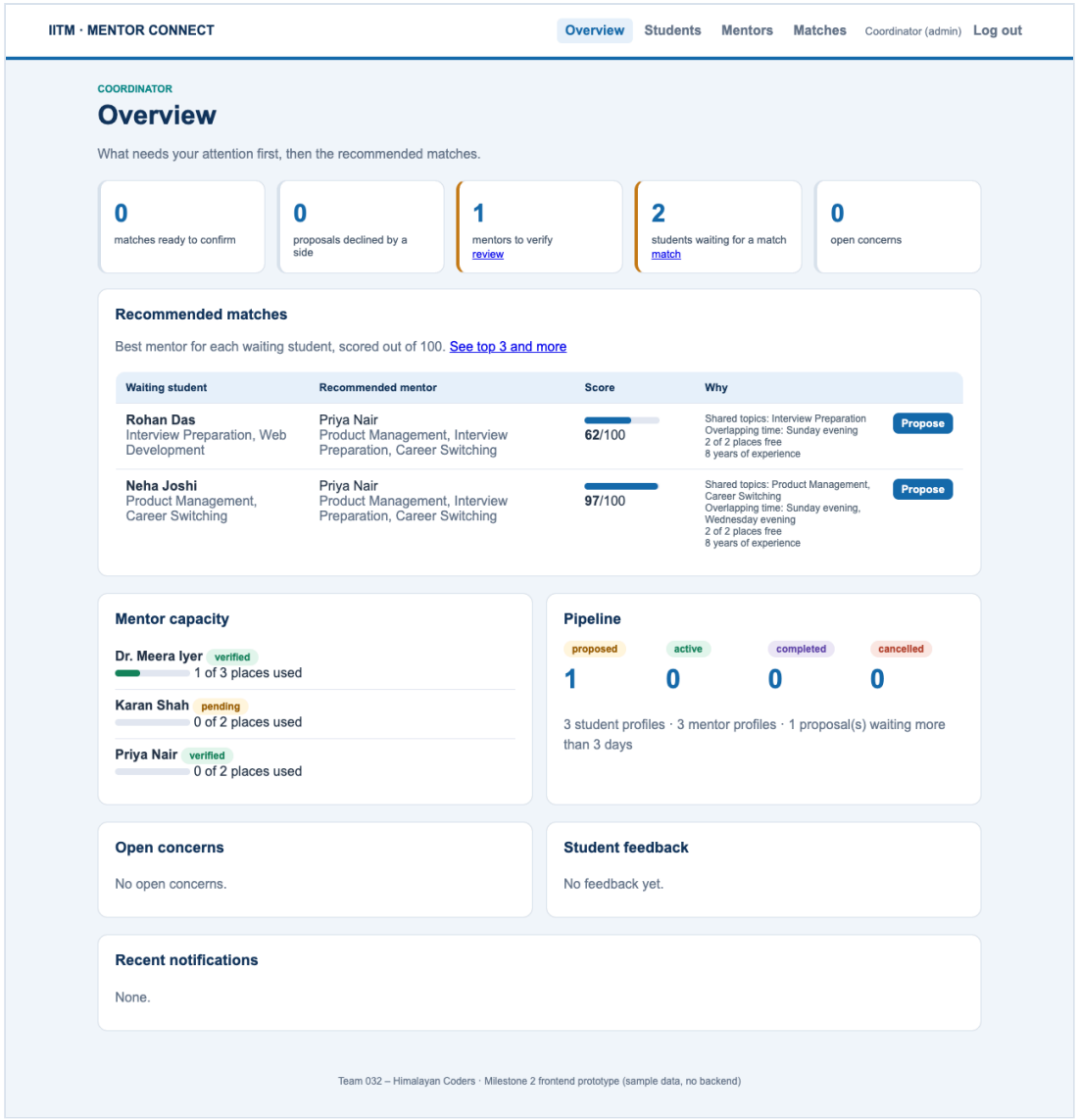


Figure 9. Coordinator overview: action cards, recommended matches with scores and reasons, mentor capacity and pipeline.

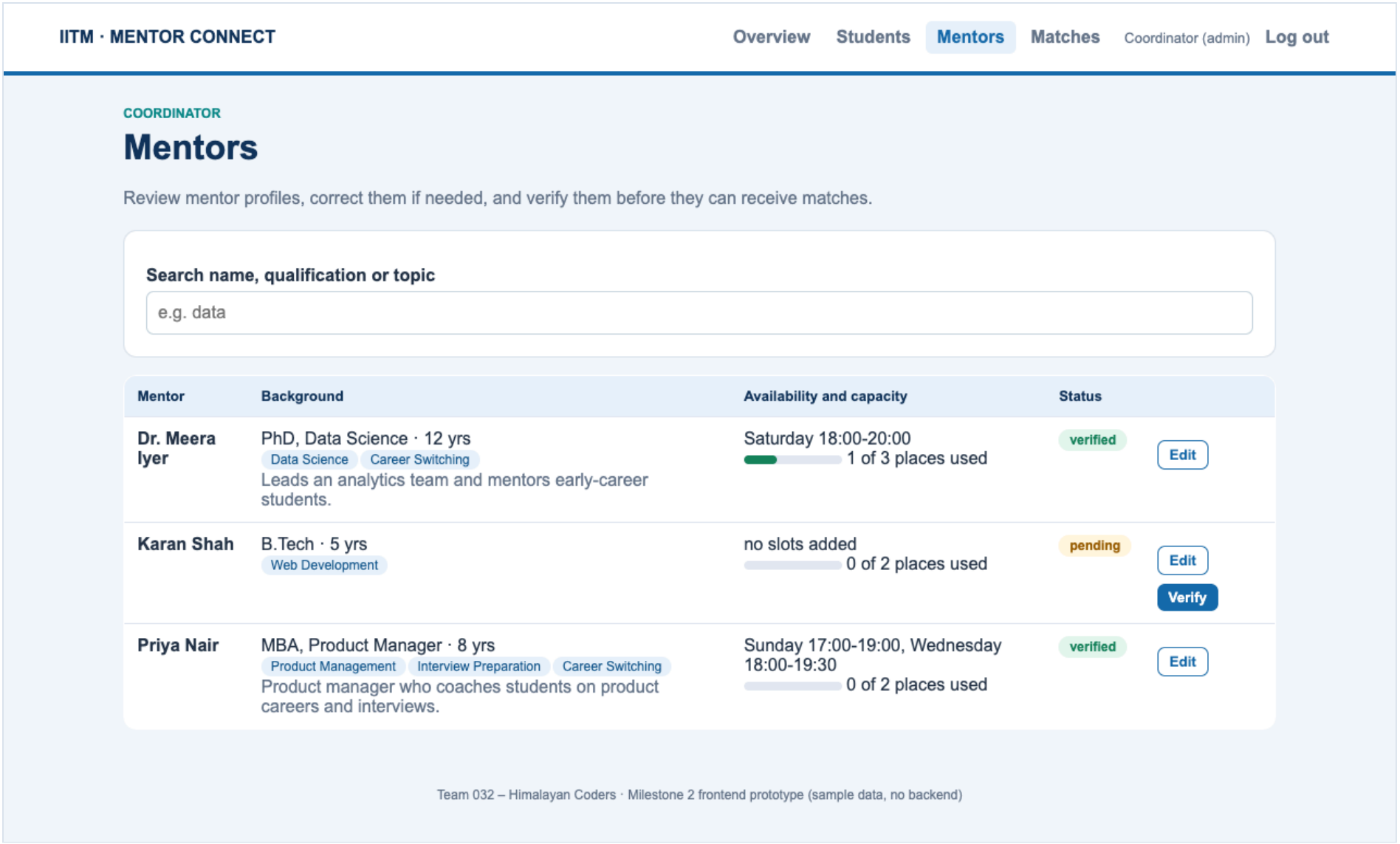


Figure 10. Mentor review with topics, availability and capacity bars.

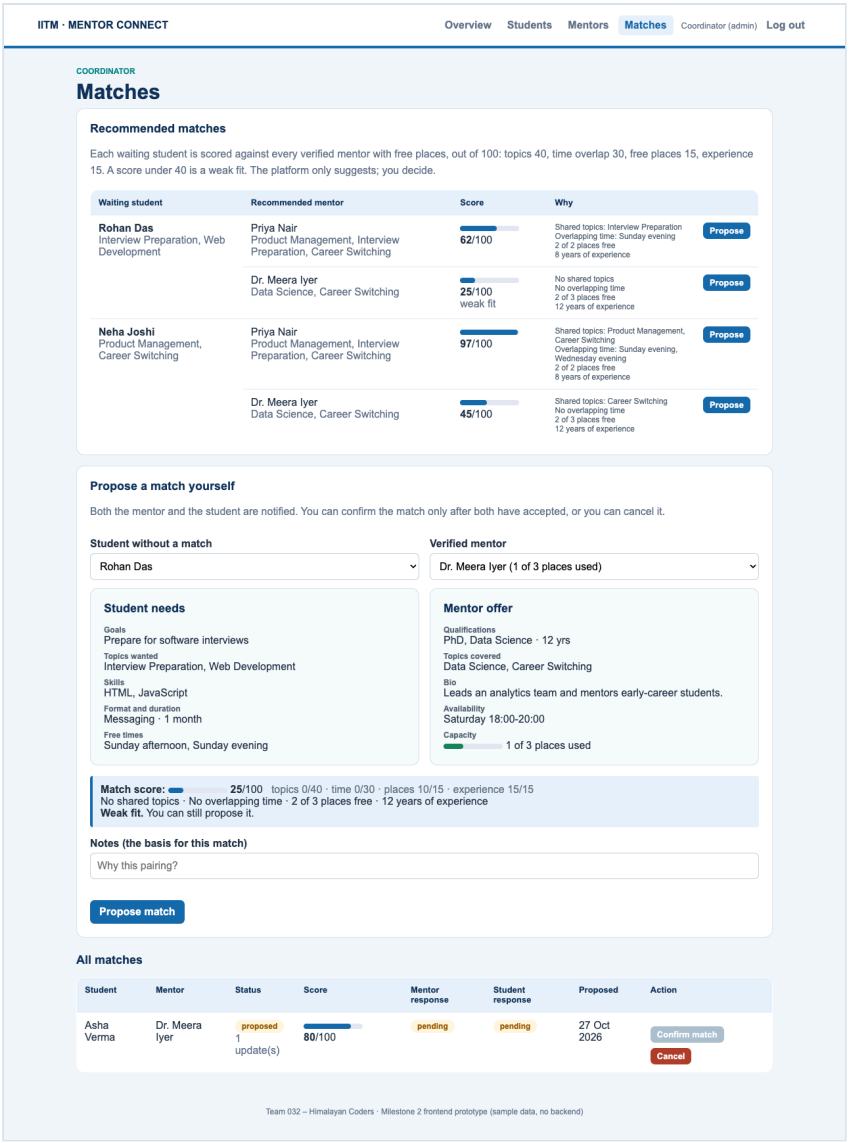


Figure 11. Matches page: top three recommended mentors per waiting student, the score of a chosen pair, and the match table with scores.

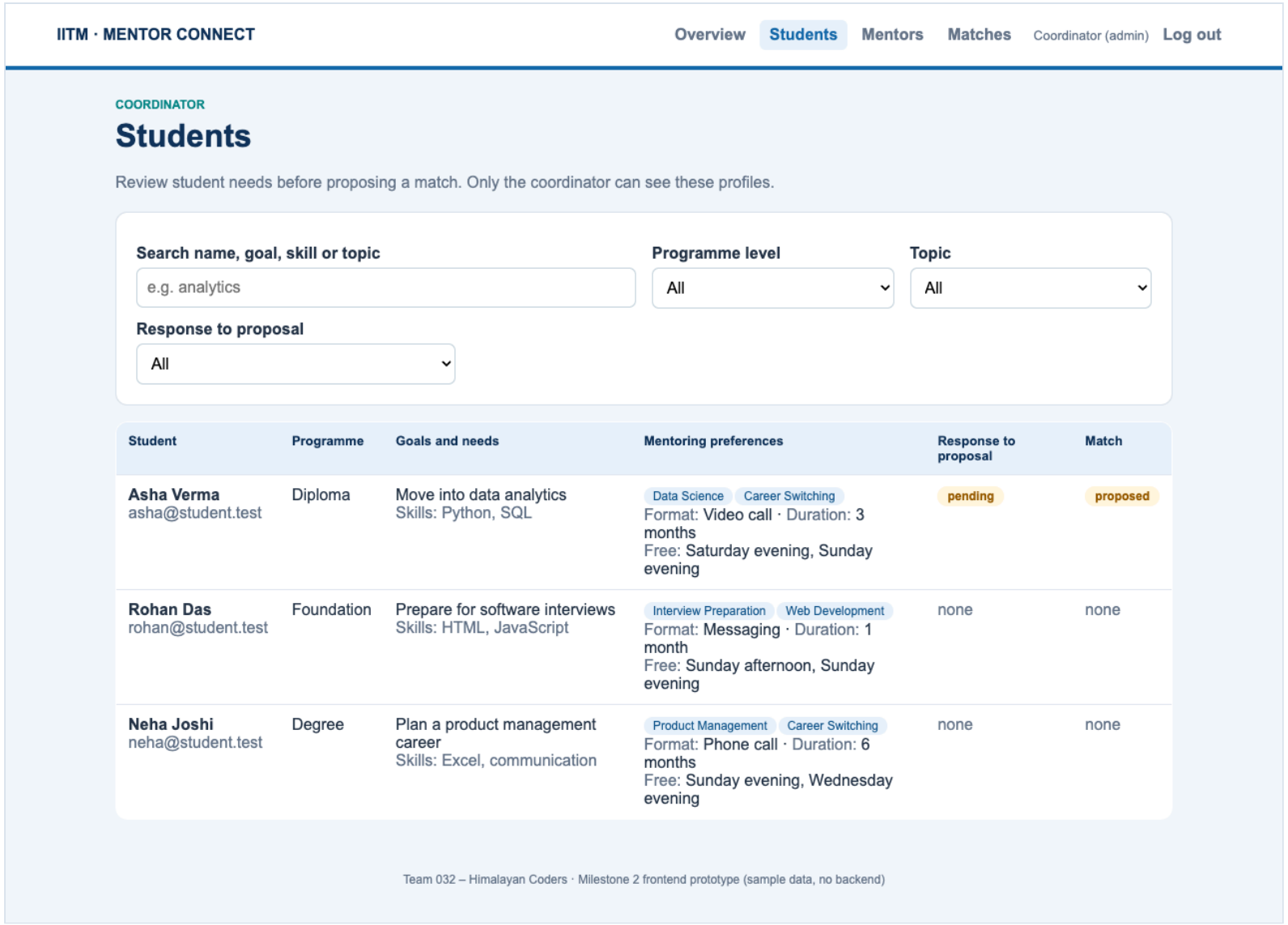


Figure 12. Coordinator view of student profiles with search and filters.

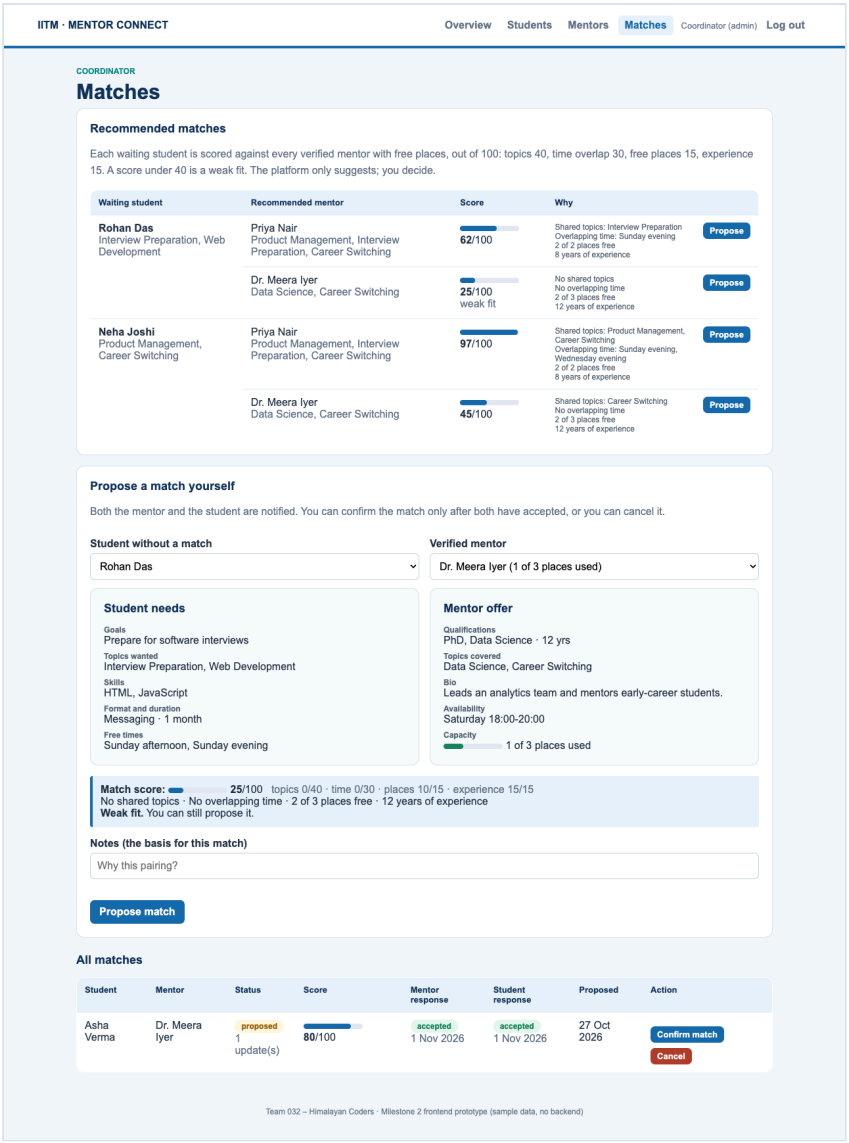


Figure 13. Coordinator view after both sides accepted: the Confirm match button is enabled. Before both accept it stays disabled.

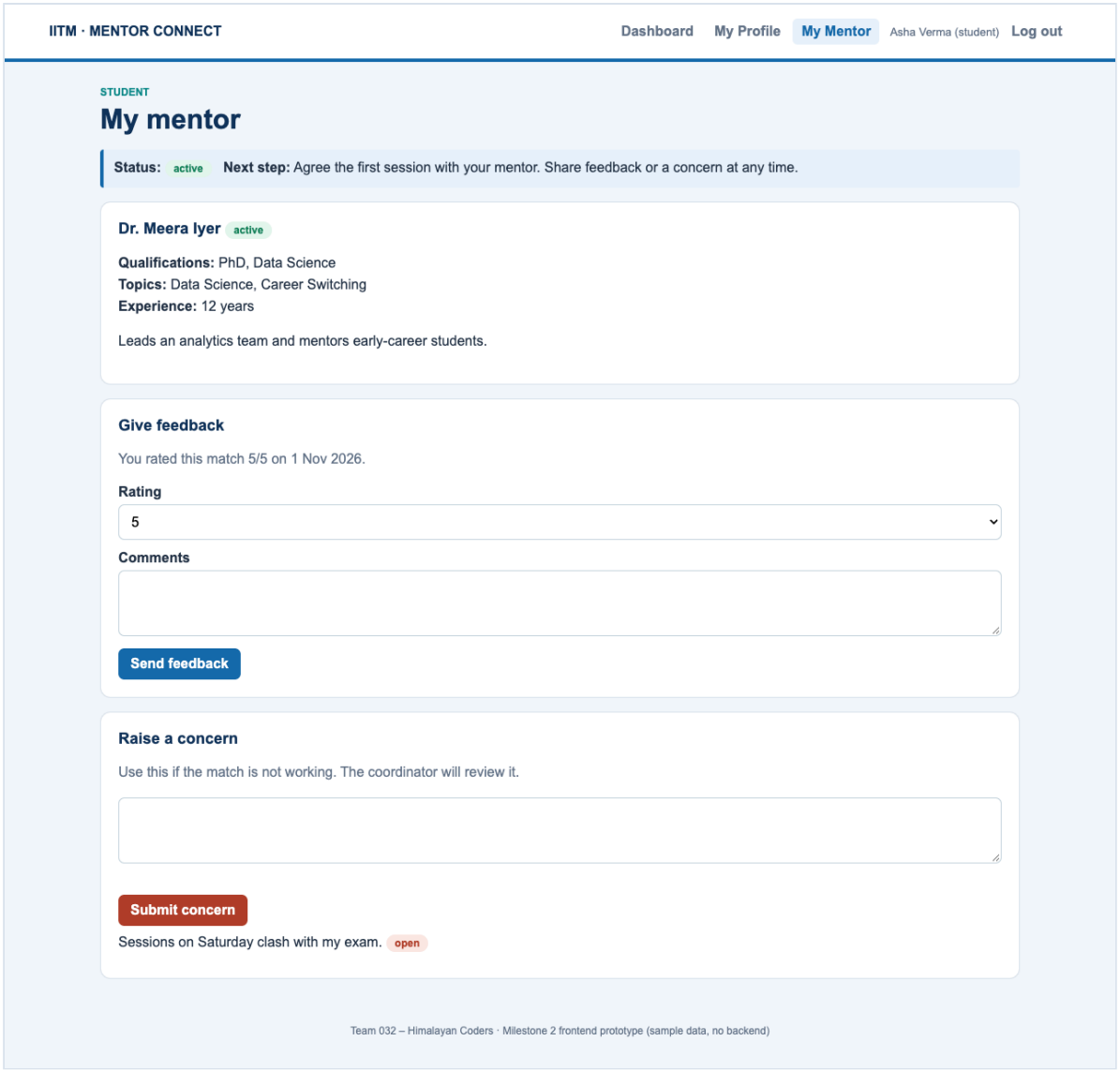


Figure 14. Student's mentor page after the coordinator confirms the match: status, next step, mentor details, feedback and concern forms.

7. Frontend Code and README

The frontend ZIP contains the 11 pages, `styles.css`, `app.js` and a README. To run it, unzip, open the `frontend` folder and open `index.html` in a modern browser. The demo password is `demo123` for the student, mentor and coordinator accounts listed on the login page.

8. Open Questions and Risks

- What happens after a mentor or a student declines a proposal (the prototype leaves the decision to the coordinator, who cancels the match and can propose a new one).
- Which channels are used for notifications, and whether email is needed.
- How long student and mentor data is kept, and who may see feedback.
- Time zone handling for availability slots.
- Whether the system should set an upper limit on the number of students per mentor, and who maintains the shared list of topics.
- Whether the score weights (topics 40, time 30, places 15, experience 15) match what the client considers important.